

# Post-Laboratory Report - #8 : Motor Position Control System

Name: Göktuğ Arda Gök

No: 30975

## Introduction

The purpose of this laboratory exercise was to understand and implement a motor position control system using servo motors, a Beckhoff PLC system, and TwinCAT software. Through this exercise, we gained hands-on experience in programming motor control, configuring visualization interfaces, and analyzing results obtained from actual experiments. The lab also provided insight into how programming and automation are integrated for precise motion control in industrial systems.

---

## Experimental Setup

### 1. Hardware Components:

- Two servo motors mounted securely in a frame.
- A Beckhoff servo motor driver for controlling motor actions.
- Beckhoff touch screen HMI for user interface.
- Beckhoff PLC for logic control.

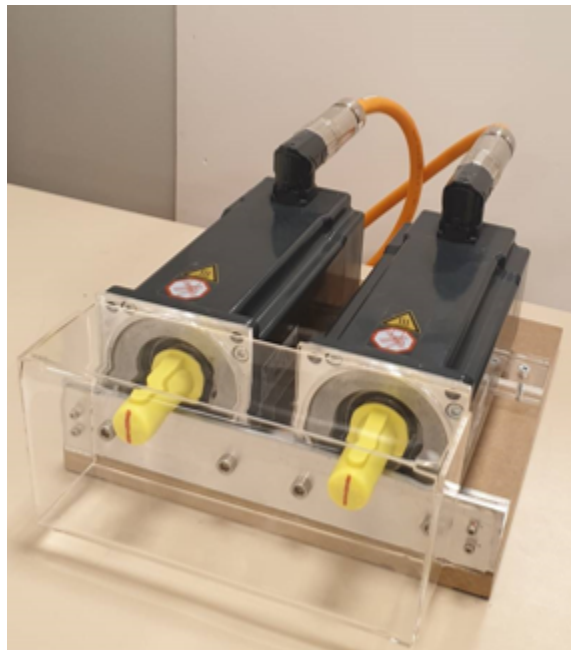


Figure 1: Servo Motors

## 2. Software Components:

- TwinCAT software used to create and execute control programs.
- Visualization tools for monitoring and user interaction.

## 3. System Objectives:

- Enable and disable motor control.
- Set and monitor reference positions for the motor shaft.
- Execute motion commands and observe the motor's actual position.

### Code Explanation

#### 1. Rising Edge Detection for Execution

h001\_firstaxisexe\_execute\_button\_pressed\_rising\_edge

(CLK:=h001\_firstaxisexe\_execute\_button\_pressed);

IF h001\_firstaxisexe\_execute\_button\_pressed\_rising\_edge.Q THEN

First\_Axis\_Move\_AbsoluteExecute := TRUE;

END\_IF

- **Purpose:** This code detects when the EXECUTE button on the HMI is pressed.
- **How it Works:**
  - A rising edge trigger is used to monitor the signal (CLK).
  - When the button is pressed (rising\_edge.Q becomes true), the motion command First\_Axis\_Move\_AbsoluteExecute is activated, enabling the motor to start moving.

#### 2. Motion Completion Logic

h001\_firstaxisexe\_done\_rising\_edge

(CLK:=First\_Axis\_Move\_AbsoluteDone AND NOT  
h001\_firstaxisexe\_execute\_button\_pressed);

```
IF h001_firstaxisexe_done_rising_edge.Q THEN
```

```
    First_Axis_Move_AbsoluteExecute := FALSE;
```

```
END_IF
```

- **Purpose:** Stops the motion command after the motor has reached its target position.
- **How it Works:**
  - The rising edge trigger monitors the completion flag (First\_Axis\_Move\_AbsoluteDone).
  - Once the motor reaches the reference position and no additional commands are being sent, the execution signal (First\_Axis\_Move\_AbsoluteExecute) is turned off.

### 3. Variable Declarations

```
VAR
```

```
First_Axis_Move_Absolute: MC_MoveAbsolute;
```

```
First_Axis_Move_AbsoluteExecute: BOOL;
```

```
First_Axis_Move_AbsolutePosition: LREAL;
```

```
First_Axis_Move_AbsoluteVelocity: LREAL;
```

```
First_Axis_Move_AbsoluteAcceleration: LREAL;
```

```
First_Axis_Move_AbsoluteDeceleration: LREAL;
```

```
First_Axis_Move_AbsoluteJerk: LREAL;
```

```
First_Axis_Move_AbsoluteDone: BOOL;
```

- **Purpose:** Defines the key variables required for controlling motor actions.
- **Variables Explained:**
  - First\_Axis\_Move\_Absolute: Main block for executing absolute position movements.
  - First\_Axis\_Move\_AbsolutePosition: Target position for the motor.

- First\_Axis\_Move\_AbsoluteVelocity: Speed at which the motor moves.
- First\_Axis\_Move\_AbsoluteAcceleration and Deceleration: Define the rate of speed increase and decrease.
- First\_Axis\_Move\_AbsoluteJerk: Smoothens motion by reducing sudden changes in acceleration.
- First\_Axis\_Move\_AbsoluteDone: A flag indicating motion completion.

#### 4. Main Control Logic

First\_Axis\_Move\_Absolute

```
(
    Execute := First_Axis_Move_AbsoluteExecute,
    Position := First_Axis_Move_AbsolutePosition,
    Velocity := First_Axis_Move_AbsoluteVelocity,
    Acceleration := First_Axis_Move_AbsoluteAcceleration,
    Deceleration := First_Axis_Move_AbsoluteDeceleration,
    Jerk := First_Axis_Move_AbsoluteJerk,
    Axis := First_Axis
);
```

First\_Axis\_Move\_AbsoluteDone := First\_Axis\_Move\_Absolute.Done;

- **Purpose:** Sends motion commands to the motor and monitors execution.
- **How it Works:**
  - Links the user-defined parameters (e.g., position, velocity) to the MC\_MoveAbsolute block.
  - The Execute signal initiates the motor's movement.
  - The Done flag indicates when the movement is completed.

## **Important Steps in the Lab**

### **1. Configuring the System:**

- Connected the servo motors to the Beckhoff driver.
- Initialized the TwinCAT environment and created a new project.
- Defined the motor parameters, including acceleration, velocity, and jerk.

### **2. Developing the Control Logic:**

- Wrote PLC code to control the motor using rising edge triggers.
- Programmed conditions for enabling motion, setting reference positions, and monitoring execution status. (figure 5 - 6)

### **3. Visualization Setup:**

- Designed a user interface with buttons for enabling/disabling, setting reference positions, and executing commands.
- Linked variables in the visualization interface with the control logic in TwinCAT.

### **4. Testing and Validation:**

- Observed the motor's behavior during test movements.
- Verified that the motor accurately moved to the set reference position and stopped correctly.

## **Experimental Results**

### **1. Visualization Observations:**

- The touch interface allowed smooth interaction. Users could enable the motor, input reference positions, and execute motion commands seamlessly. (figure 2)
- The status box turned green when the motor was enabled, providing a clear visual cue of system readiness. (figure 3)

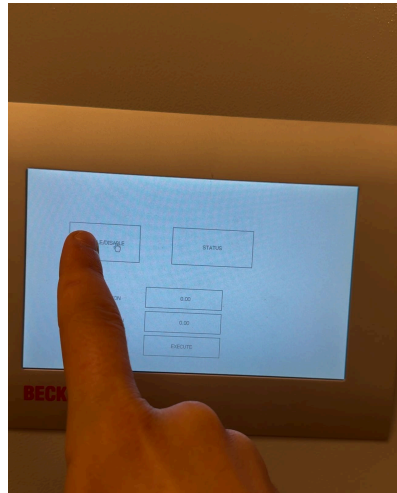


Figure 2: HMI Screen, Start System

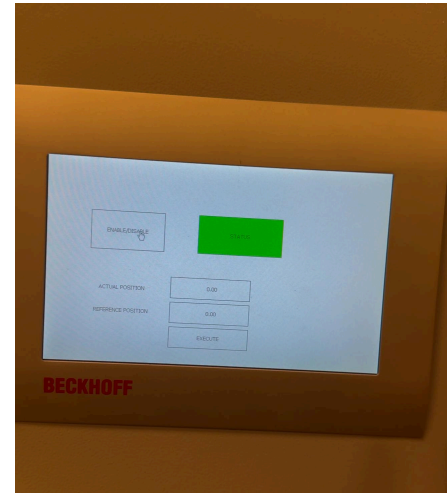


Figure 3: System Enabled, Ready to Work

## 2. Motor Behavior:

- Reference position set to 150 mm. The motor rotated accurately, and the actual position matched the reference. (figure 4)
- Rapid position changes between -500 mm and 500 mm showed no errors, and the motor returned to its original angle as expected.
- Continuous execution validated the reliability of the control logic.

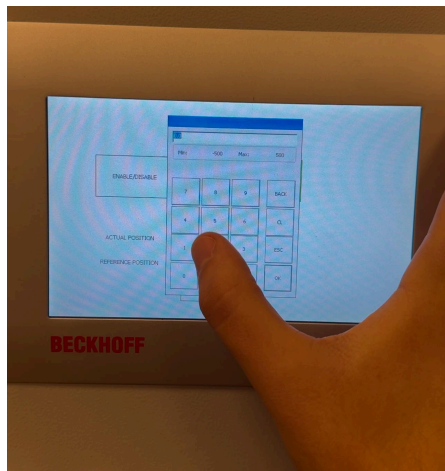


Figure 4: Entering Reference Point on Numpad

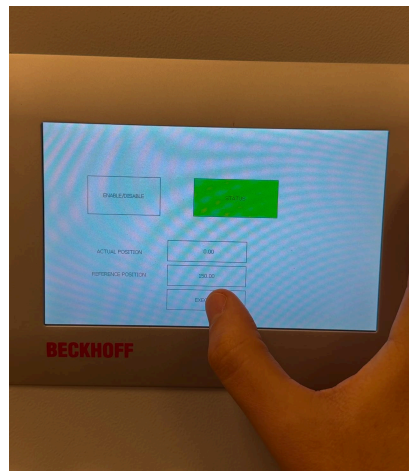


Figure 5: Executing Written Motor Position

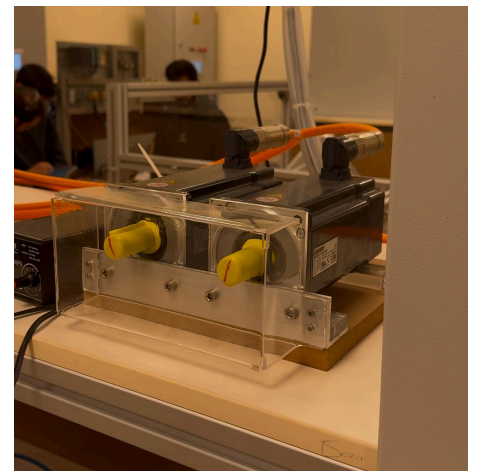


Figure 6: Left Motor Starts Motion Till Reaching Reference Position

### 3. HMI:

- Photos of the HMI and motor setup illustrate the practical implementation. The interface design with labeled buttons (ENABLE/DISABLE, EXECUTE) and numeric input fields ensured user-friendliness. (figure 7)

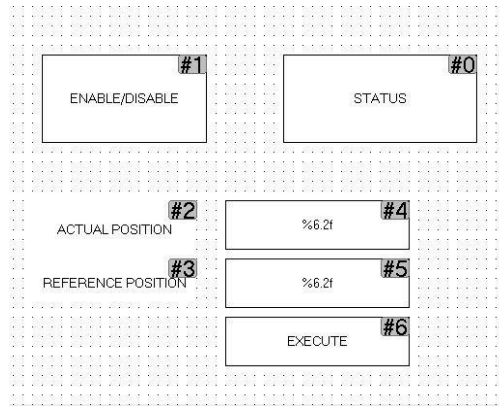


Figure 7: HMI Formation on TwinCAT

### 4. Key Results:

- Precise control over motor positions was achieved.
- Visualization provided a robust feedback mechanism for monitoring and adjusting motor states.

## Discussion

### 1. Analysis of Steps and Observations:

- The **Enable/Disable button** served as a fundamental control, ensuring that the motor was not accidentally activated. The seamless transition between states on the HMI verified its reliability.
- The **reference position input** allowed dynamic target setting. Experimentally, positions like 150 mm and rapid changes between -500 mm and 500 mm showed consistent accuracy in motor movement.
- The **execution button** linked to the rising edge trigger demonstrated smooth initiation of motion commands. Observations confirmed precise movements, and the motor always returned to the initial position upon task completion. (figure 5)

- Status feedback and real-time position monitoring on the HMI added a layer of safety and clarity, enabling operators to track system performance without ambiguity.

## 2. Successes:

- **Accuracy:** The motor consistently reached the desired positions within expected tolerances, validating the logic and parameter configurations.
- **System Responsiveness:** The commands executed promptly upon button presses, showcasing the robustness of the TwinCAT logic and hardware integration.
- **User-Friendly Interface:** The design of the HMI allowed easy navigation and input, making it suitable for real-world industrial applications.

## 3. Educational Insights:

- This lab reinforced critical concepts of motion control, including the practical use of rising edge triggers, feedback loops, and motor parameter optimization.
- Hands-on interaction with TwinCAT programming provided a valuable opportunity to understand industrial automation workflows, bridging the gap between theoretical knowledge and practical application.
- Observing the direct impact of HMI interactions on motor performance emphasized the importance of user interface design in industrial systems.

## Conclusion

This laboratory exercise demonstrated the seamless integration of hardware and software for industrial motor control. By programming motion control using TwinCAT and PLC logic, we achieved accurate and reliable motor movements. Observations confirmed the effectiveness of the control logic and the visualization interface, highlighting the importance of careful parameter tuning and system feedback.